

# ENHANCED ASR FOR STUTTERING SPEECH: COMBINING ADVERSARIAL AND SIGNAL-BASED DATA AUGMENTATION

Shangkun Huang, Dejun Zhang, Jing Deng, Rong Zheng

Beijing Fosafer Information Technology Co., Ltd.

## ABSTRACT

This paper presents our submission to the SLT2024 StutteringSpeech Challenge, focusing on augmenting stuttering data using straightforward and effective techniques. We combined adversarial and signal-based data augmentation methods, including modifying speech rate and rhythm, inserting silence segments, repeating speech segments, and applying Generative Adversarial Network-based (GAN-based) perturbation. These techniques enabled us to generate stuttering speech from fluent speech, which we used to train our automatic speech recognition (ASR) model, enhancing its robustness for individuals with stuttering. Our system achieved a character error rate (CER) of 12.30% in the StutteringSpeech Challenge Track 2, demonstrating a relative improvement of 35.87% over the official baseline and securing first place in the competition.

**Index Terms**— Stuttered speech, automatic speech recognition, GAN, data augmentation method

## 1. INTRODUCTION

Stuttering is a multifaceted neurodevelopmental disorder affecting over 80 million people worldwide [1]. Common manifestations include word, phrase, or sound repetitions, silent pauses (blocking), elongation of specific syllables or words (prolongation), and interjections. Stuttering hinders daily verbal communication for people with stuttering (PWS) and causes stress, shame, and feelings of inferiority. This fear of speaking can lead to self-isolation [2] [3]. In mainland China, speech therapy practice is still in its infancy compared to North America and Western Europe, leading to delays in diagnosing and treating stuttering in children. The SLT 2024 Mandarin Stuttering Event Detection and ASR Challenge<sup>1</sup> (StutteringSpeech Challenge) aims to reduce communication obstacles for PWS, address social discrimination, encourage inclusive research, and advance speech technologies tailored to the needs of individuals who stutter.

Advances in deep learning have facilitated the development of ASR technology, with corresponding voice assistants

incorporated into various commercial products. However, recognizing stuttering speech accurately remains challenging [4–7] due to several factors, including the scarcity of stuttering data caused by privacy and sensitivity concerns, making it difficult to amass a substantial dataset for ASR system development. Additionally, the distinct characteristics of stuttering speech, such as repetitive, delayed and blocked behaviors in PWS, create a substantial mismatch with fluent speech. Furthermore, there needs to be more recognition regarding the necessity of developing a dedicated system explicitly tailored for PWS [8]. The lack of sufficient stuttering speech data has been a significant bottleneck in addressing ASR systems' performance differences [9] [10] [11]. Consequently, the utilization of data augmentation methods is crucial in mitigating and resolving the issue of limited data availability for stuttering speech recognition. Various data augmentation techniques, including tempo and speed perturbations [12], SpecAugment [13], simulated noise, reverberant speech [14], GAN-based adversarial methods [15–17], speech synthesis [18], and voice conversion [19] [20], have been thoroughly investigated in conventional speech recognition applications.

While research focused on directly improving ASR systems for stuttering speech is limited, some advancements have been achieved. Zayats et al. [21] and Shonibare et al. [22] improved ASR performance by categorizing stuttering events. Lea et al. [23] fine-tuned an ASR model using data from stutterers and improved voice activity detection (VAD) techniques to enhance ASR performance. Mitra et al. [24] reduced the word error rate (WER) by adjusting ASR decoding parameters. Zhang et al. [7] optimized an ASR model by generating stuttering speech using text-to-speech (TTS) techniques, demonstrating the viability of their approach. Limited research has been conducted on enhancing stuttering speech data, as it necessitates a comprehensive comprehension of the stuttering speech patterns.

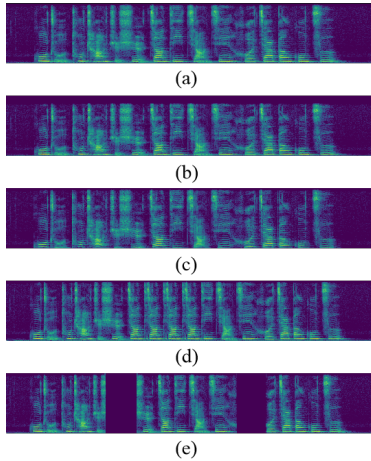
Our study introduces a straightforward and effective multi-stage data augmentation method to improve the performance of stuttering ASR by analyzing stuttering speech patterns and focusing on the articulatory characteristics of PWS. Various data augmentation methods are thoroughly examined and contrasted to increase the diversity of training data and enhance the robustness of the ASR model to accom-

<sup>1</sup>This work is supported by the National Key Research and Development Program of China (No.2022YFF0608504).

<sup>1</sup><https://stutteringspeech.org>

moderate limited stuttering speech samples better. Algorithms simulating stuttering speech from fluent speech, including Speed Perturbation, Tempo Perturbation, InsertSilence, RepeatPart, and SpecAugment, are proposed to simulate overall rhythm, speech rate, and spectral envelope changes. Furthermore, inspired by [25] [26] to capture more fine-grained patterns, fine-grained spectral-temporal features associated with stuttering speech are also injected into the speech data augmented by GAN-based methods such as Parallel WaveGAN (PWG) [27], BigVGAN [28], and Voces [29].

The main contributions of this paper are summarized as follows. Firstly, we present, for the first time, a set of simple and efficient methods combining adversarial and signal-based data augmentation specifically designed to enhance the training process of ASR for stuttering. This includes a comparative study of various speech perturbation and adversarial data augmentation methods. Secondly, our method applied to the current state-of-the-art ASR model E-branchformer, achieved a 12.01% CER on the best AS-70 test set, using only the Aishell-1 and AS-70 datasets.



**Fig. 1:** Example spectrogram of (a) Fluent, (b) Speed Perturbation, (c) Tempo Perturbation, (d) RepeatPart, and (e) InsertSilence.

## 2. SIGNAL-BASED DATA AUGMENTATION

This section introduces five signal-based front-end data augmentation methods tailored for stuttering speech recognition. These methods include traditional perturbation (speed perturbation and tempo perturbation), as well as the InsertSilence and RepeatPart methods, which address the distinctive blocking and repetitive articulation characteristics of stuttering. We also discuss the dynamic SpecAugment technique. These approaches adjust rhythm, speech rate, and spectral envelope for fluent speech, forming the basis for training GAN-based augmentation models in Section 3. Fig. 1, illustrates the spectral changes in fluent speech processed by these methods.

### 2.1. Speed Perturbation

Speed perturbation [30] enhances speech recognition performance and is efficient to implement. It involves adjusting the time-domain speech signal  $x(t)$  by altering the sampling resolution with a perturbation factor  $\alpha$  resulting in a velocity-perturbed signal  $y(t) = x(\alpha t)$ . This adjustment in the time domain corresponds to conducting a similar operation in the frequency domain.

$$X(f) \rightarrow \frac{1}{\alpha} X\left(\frac{1}{\alpha} f\right) \quad (1)$$

where  $X(f)$  and  $\frac{1}{\alpha} X(\frac{1}{\alpha} f)$  denote the Fourier transforms of  $x(t)$  and  $y(t)$ , respectively. However, it is essential to note that velocity perturbation changes the audio duration and overall spectral envelope of the input audio, affecting the pitch and rhythm of the original speech, as it involves resampling the original speech signal. This study systematically introduced a fixed set of perturbation factors  $\{0.9, 1.0, 1.1\}$  to speech samples from individuals with and without stuttering. The objective was to replicate variations in speech rate and volume observed in PWS, thereby simulating their speech characteristics.

### 2.2. Tempo Perturbation

Tempo perturbation modifies the duration of the input time-domain signal  $x(t)$  while keeping its pitch and overall contour unchanged [31]. The WSOLA algorithm [32] is commonly used to change the duration of the time-domain signal by a factor  $R_2$  while ensuring that the pitch and spectrum of the signal remain unchanged. The time-domain input speech signal  $x(t)$  is decomposed into equally spaced analytic blocks  $\tilde{x}_m(r)$  along the time axis. For a given perturbation factor  $R_2$ , the synthesized block  $\tilde{y}(r)$  is relocated by  $H_s$  (synthesis hypothesis) according to the following equation.

$$H_s = \alpha \times H_\alpha \quad (2)$$

When stuttering, PWS generally has a slower speech cadence than fluent speech. We modified the parameter range of fluent speech rhythm to  $[0.7, 1.0]$  to match the different speech rhythms of different speakers in the stuttering speech corpus.

### 2.3. InsertSilence

We developed the “InsertSilence” algorithm to simulate blocking behavior in stuttering. This method randomly determines the number, position, and duration of silent inserts within specified ranges. Typically, 1 to 3 silent inserts with durations from 100 to 600 milliseconds are added at designated positions in the audio. This introduces random pauses and delays into fluent speech, emulating the pauses characteristic of stuttering. Algorithm 1 provides implementation details.

---

**Algorithm 1** InsertSilence

---

**Require:**  $a$ : audio,  $cr$ : silence count range,  $lr$ : silence length range

**Ensure:** modified audio

```
1:  $n \leftarrow \text{Random}(cr)$ 
2:  $s \leftarrow$  empty list
3: for  $i \leftarrow 1$  to  $n$  do
4:   Append ( $\text{Random}(0, \text{Length}(a)), \text{Random}(lr)$ ) to  $s$ 
5: end for
6: Sort  $s$  by position in descending order
7: for each  $(p, l)$  in  $s$  do
8:   samples  $\leftarrow \text{ConvertToSamples}(l, sr)$ 
9:    $a \leftarrow \text{Insert}(a, \text{CreateZeroArray}(\text{samples}), p)$ 
10: end for
11: return  $a$ 
```

---

## 2.4. RepeatPart

The RepeatPart feature of the audiomentations<sup>2</sup> toolkit was used to simulate the repetition of words, phrases, and sounds in stuttering by repeating audio segments multiple times. The number of repetitions ranges from 1 to 3, and the duration of each repeated part ranges from 200 to 600 milliseconds. We used cross-fade processing with a cross-time parameter of 0.02 seconds to ensure seamless integration of repetitions with the original audio, effectively mimicking the repetitive patterns of stuttering.

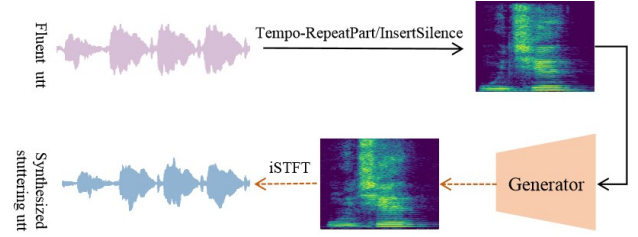
## 2.5. SpecAugment

SpecAugment [13] is a dynamic data augmentation technique that can be directly applied to spectral speech features for DNN training. The enhancement strategy aims to train more generalized ASR models by predicting data changes in the temporal direction, partial information loss in the frequency direction, and loss of small speech fragments. To achieve this, masks are constructed to dynamically block or modify information in the time and frequency directions. The width and position of the masks are randomly determined, ensuring that the DNN encounters different versions of the input speech in each training period. We enabled time warping with a window size of 5 using bicubic interpolation; frequency masking with a width range of 0 to 27, applied twice; and time masking with a width ratio range of 0% to 5%, applied ten times.

## 3. GAN BASED DATA AUGMENTATION

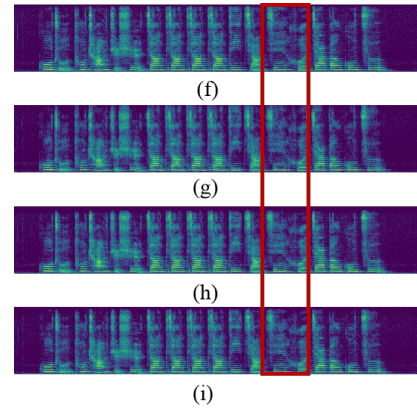
Current signal-based data augmentation methods struggle to accurately capture the subtle distinctions between fluent speech and stuttering speech. While methods such as RepeatPart and InsertSilence effectively characterize stuttering repetitions and blocking behaviors, they fall short in addressing

<sup>2</sup><https://github.com/iver56/audiomentations.git>



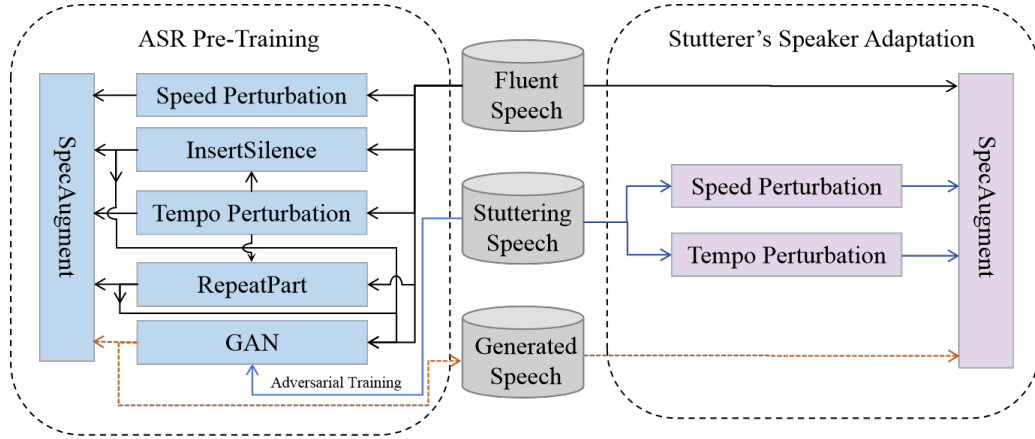
**Fig. 2:** GAN-based speech synthesized for stuttering using tempo-RepeatPart/InsertSilence

other nuanced features of stuttering speech. To address this challenge, our study explores using GANs to represent better the detailed spectral-temporal variances between stuttering and fluent speech. The signal-based data augmentation methods discussed in Section 2 are limited to capturing general characteristics like speech rate, speech rate, and spectral envelope variations in PWS. In this section, we employ various GAN models, including Parallel WaveGAN, BigVGAN, and Vocos, to learn more fine-grained spectral-temporal features of stuttering speech.



**Fig. 3:** Example spectrogram of (f) Tempo-RepeatPart, (g) Tempo-RepeatPart-PWG, (h)Tempo-RepeatPart-BigVGAN, and (i)Tempo-RepeatPart-Vocos.

GAN has been proposed by Goodfellow et al. [33], consisting of two components: a generator ( $G$ ) and a discriminator ( $D$ ). The generator ( $G$ ) learns the data distribution  $p_{\text{data}}(x)$  by generating samples  $\hat{x} = G(z)$  from a noise source  $z$ , aiming to create indistinguishable data to deceive the discriminator  $D$ . The discriminator  $D$  distinguishes between real data  $x$  and generated data  $\hat{x}$ . This adversarial training minimizes the Jensen-Shannon divergence between the real data distribution and the model data distribution. The interaction guides  $G$  to synthesize coherent waveforms with realistic stuttering characteristics while minimizing perceptual artifacts detectable by human ears. The GAN training process involves minimizing the following adversarial loss:



**Fig. 4:** Overview of a data augmentation method designed for stuttering speech recognition; **Fluent speech** represents the Aishell-1 dataset; **Stuttering speech** represents the AS-70 dataset; and **generated speech** represents stuttering speech generated using GANs. The solid black line represents fluent speech, the solid blue line represents stuttered speech, and the dashed yellow line represents speech generated by GANs.

$$\mathcal{L}_{GAN}(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (3)$$

We first train the GAN using stuttering speech data. Then, we use the fluent speech processed by the signal-based augmentation method for inference in the GAN, generating stuttering speech (see Fig. 2). To fully explore the potential of GAN-based augmentation, we train Parallel WaveGAN, Vocos, and BigVGAN. These models, with their unique approaches, help us deeply investigate GAN-based augmentation for stuttering speech recognition. Vocos eliminates transposed convolutions for upsampling and estimates phase angles during training. In contrast, BigVGAN scales up model parameters and introduces periodic activations into the generator.

BigVGAN, utilizing anti-aliasing filters, can recover some upper frequency harmonics. However, Vocos demonstrates a more precise reconstruction of these harmonics without requiring additional modules. Fig. 3, illustrates the spectral variation of stuttering speech generated by these three GAN models.

## 4. EXPERIMENTAL SETUP

### 4.1. Data

The ASR model training utilized datasets from Aishell-1 [34] and AS-70 [8]. The Aishell-1 dataset was used for the initial training phase, employing diverse data augmentation techniques. In contrast, the AS-70 dataset was used for stutterer's speaker adaptation and GAN training, focusing on stutterers. AS-70 includes dialogue and command dictation scenarios, featuring roughly thirty-minute interviews and predetermined

command word articulations by 72 native Mandarin PWS, with a total duration of approximately 50 hours.

Different data augmentation strategies were employed for various ASR systems, as detailed in Table 1. This table provides a breakdown of data augmentation methods and volumes for both the ASR pre-training and the stutterer's speaker adaptation stages. Fig. 4 further illustrates the sequence and application of each data augmentation technique.

### 4.2. Adversarial training

We analyzed the effect of GAN-based augmentation on stuttering speech ASR using a subset of the stutter challenge dataset. This subset comprised 10,963 utterances, divided into 9,963 for training and 1,000 for validation. Despite differences in fundamental frequency (F0) between male and female speakers, the subset was not split by gender due to the number of utterances. The same split was used to train PWG, BigVGAN, and Vocos.

For PWG, the generator and discriminator were trained for 200k steps, with a learning rate of 0.0001 for the generator and 0.00005 for the discriminator, both reduced by half during training. BigVGAN training lasted 30k epochs, using multi-period and multi-resolution discriminators to supervise waveform generation. The learning rates for the generator and discriminator started at 0.0001 and were adjusted using an exponential decay method. Vocos utilized a ConvNeXt backbone for its generator and multi-period and multi-resolution discriminators. It was trained for 30k steps with an initial learning rate of 0.0005, optimized by AdamW, and adjusted using a cosine schedule with a warmup strategy. For fair comparison, the training procedures followed the original configurations specified in the respective papers.



**Table 1: ASR Pre-Training and Stutterer’s Speaker Adaptation Breakdown with Data Augmentation**

| System | Stage                                | Data Augmentation                                                                                                                 | Hours |
|--------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------|
| S0     | ASR<br>Pre-Training                  | <i>SetA</i> : -                                                                                                                   | 178   |
| S1     |                                      | <i>SetA</i> : $SP \times 3$                                                                                                       | 534   |
| S2     |                                      | <i>SetA</i> : $TP \times 3$                                                                                                       | 534   |
| S3     |                                      | <i>SetA</i> : $IS \times 3$                                                                                                       | 534   |
| S4     |                                      | <i>SetA</i> : $RP \times 3$                                                                                                       | 534   |
| S5     |                                      | <i>SetA</i> : Parallel WaveGan $\times 3$                                                                                         | 534   |
| S6     |                                      | <i>SetA</i> : $SP \times 3 + TP \& IS \times 2$                                                                                   | 890   |
| S7     |                                      | <i>SetA</i> : $SP \times 3 + TP \& RP \times 2$                                                                                   | 890   |
| S8     |                                      | <i>SetA</i> : $SP \times 3 + TP \& RP \& IS \times 2$                                                                             | 890   |
| S9     |                                      | <i>SetA</i> : $SP \times 3 + TP \& IS \times 2 + TP \& RP \times 2$                                                               | 1246  |
| S10    |                                      | S9-part + $TP \& RP \& IS \& \text{Parallel WaveGan} \times 1 + TP \& IS \times 2 + TP \& RP \& \text{Parallel WaveGan} \times 1$ | 1443  |
| S11    |                                      | S9-part + $TP \& RP \& IS \& \text{BigVGAN} \times 1 + TP \& IS \times 2 + TP \& RP \& \text{BigVGAN} \times 1$                   | 1443  |
| S12    |                                      | S9-part + $TP \& RP \& IS \& \text{Vocos} \times 1 + TP \& IS \times 2 + TP \& RP \& \text{Vocos} \times 1$                       | 1443  |
| S1-S9  | Stutterer’s<br>Speaker<br>Adaptation | <i>SetB</i> : $SP \times 3 + TP \times 3$ ; <i>SetA</i> $\times 1$                                                                | 436   |
| S10    |                                      | <i>SetB</i> : $SP \times 3 + TP \times 3$ ; <i>SetA</i> $\times 1$ ; Parallel WaveGan-part                                        | 502   |
| S11    |                                      | <i>SetB</i> : $SP \times 3 + TP \times 3$ ; <i>SetA</i> $\times 1$ ; BigVGAN-part                                                 | 502   |
| S12    |                                      | <i>SetB</i> : $SP \times 3 + TP \times 3$ ; <i>SetA</i> $\times 1$ ; Vocos-part                                                   | 502   |

*SetA*: Aishell-1; *SetB*: AS-70; *SP*: Speed perturbation  $SP \in \{0.9, 1.0, 1.1\}$ ; *TP*: Tempo perturbation  $TP \in [0.7, 1.0]$ ; *RP*: RepeatPart, *cr* is the silence count range  $cr \in [1, 3]$ , *lr* is the silence length range  $lr \in [200, 600]$  ms; *IS*: InsertSilence, *cr* is the silence count range  $cr \in [1, 3]$ , *lr* is the silence length range  $lr \in [100, 600]$  ms; &: indicates that data augmentation methods are used on top of each other;  $\times$ : indicates the multiplier of expansion after data augmentation.

### 4.3. ASR model training

The ASR training consists of two phases. The first phase involves processing fluent speech with various data augmentation techniques to pre-train the ASR models. The second phase adapts the pre-trained models to stuttering speakers using the official stuttering data and a small amount of fluent data. The specific data augmentation methods and training data used by each system are detailed in Table 1. SpecAugment, a critical augmentation method, was used in all systems, and the specific parameter configuration is referred to in Section 2.5. During the second training stage for the stutterer’s speaker adaptation, frequency masking in SpecAugment was turned off to enhance performance. Each ASR system was trained for 40 epochs in the first stage and 20 epochs in the second stage.

All experiments utilized ESPnet [35] with ASR models based on the E-Branchformer [36] architecture. The input features were 80-dimensional log Mel features, extracted with a 32ms window size and a 10ms stride. The E-Branchformer encoder comprised 17 layers with a feature dimension of 512. Self-attention had a head dimension of  $d/64$ , and the intermediate hidden dimensions of the cgMLP and FFN were  $6d$  and  $4d$ , respectively. The encoder used a dropout rate of 0.1 and swish activation functions in the FFNs and convolution modules, with a kernel size of 31. The experiments employed a 6-layer Transformer decoder with the same feature hidden dimension as the encoder and a dropout rate of 0.2. Detailed configuration parameters can be found in the referenced

repository<sup>3</sup>.

## 5. RESULTS AND DISCUSSION

### 5.1. Performance of data augmentation

As described in Section 4.3, our ASR model training consists of two stages. First, fluent speech is processed with various data augmentation techniques for pre-training the ASR model. Then, stuttered speech data is used for the stutterer’s speaker adaptation based on stage one. To demonstrate the effectiveness of our proposed data augmentation methods for stuttered speech recognition, we compare ASR performance with each data augmentation method and their combinations in Table 2.

We found that each data augmentation method reduced the ASR model’s CER to varying degrees, confirming that our methods improve the alignment of fluent and stuttered speech. System S9, trained on fluent speech data with all signal-based augmentation techniques, showed a 9.18% absolute CER improvement compared to system S0, which had no augmentation.

To further validate our approach, we employed adversarial data augmentation using Parallel WaveGAN, BigVGAN, and Vocos in systems S10-S12, respectively. Building on S9, these systems introduced stuttered speech-related spectral-temporal features into the ASR model training. System

<sup>3</sup>[https://github.com/espnet/espnet/blob/master/egs2/librispeech/asr1/conf/tuning/train\\_asr\\_e\\_branchformer.yaml](https://github.com/espnet/espnet/blob/master/egs2/librispeech/asr1/conf/tuning/train_asr_e_branchformer.yaml)

**Table 2:** Comparison of ASR Performance (CER %) with Various Data Augmentation Methods and Their Combinations for Stuttering Speech Recognition

| System   | Data Augmentation                             | Processed Fluent Speech for ASR Pre-Training |       |       |       |       |       | Stutterer's Speaker Adaptation |              |              |              |              |              |
|----------|-----------------------------------------------|----------------------------------------------|-------|-------|-------|-------|-------|--------------------------------|--------------|--------------|--------------|--------------|--------------|
|          |                                               | Mild                                         | Mod.  | Sev.  | Conv. | Cmd.  | All   | Mild                           | Mod.         | Sev.         | Conv.        | Cmd.         | All          |
| Baseline | -                                             | -                                            | -     | -     | -     | -     | -     | 17.27                          | 19.45        | 29.53        | 17.70        | 21.50        | 19.18        |
| S0       | Non-augmentation                              | 40.41                                        | 41.51 | 63.08 | 45.58 | 40.51 | 43.70 | -                              | -            | -            | -            | -            | -            |
| S1       | <i>SP</i>                                     | 35.45                                        | 36.69 | 59.88 | 41.46 | 34.90 | 39.00 | 16.28                          | 18.64        | 27.40        | 16.29        | 20.86        | 18.08        |
| S2       | <i>TP</i>                                     | 34.80                                        | 36.07 | 56.30 | 40.30 | 34.06 | 37.96 | 15.48                          | 18.00        | 26.10        | 16.27        | 18.70        | 17.33        |
| S3       | <i>IS</i>                                     | 33.18                                        | 34.60 | 54.65 | 38.24 | 33.15 | 36.35 | 13.25                          | 15.38        | 23.64        | 15.10        | 14.64        | 15.03        |
| S4       | <i>RP</i>                                     | 33.54                                        | 34.97 | 53.55 | 38.35 | 33.40 | 36.52 | 13.53                          | 16.18        | 23.17        | 14.86        | 15.63        | 15.26        |
| S5       | Parallel WaveGan                              | 34.40                                        | 35.27 | 57.77 | 40.05 | 33.94 | 37.77 | 14.36                          | 16.38        | 26.12        | 17.07        | 14.83        | 16.30        |
| S6       | <i>SP</i> + <i>TP</i> + <i>IS</i>             | 31.85                                        | 33.73 | 54.73 | 37.43 | 31.63 | 35.26 | 12.95                          | 15.03        | 22.27        | 13.78        | 15.50        | 14.45        |
| S7       | <i>SP</i> + <i>TP</i> + <i>RP</i>             | 32.36                                        | 32.80 | 53.89 | 37.06 | 32.61 | 35.42 | 12.95                          | 14.21        | 21.39        | 13.82        | 14.90        | 14.24        |
| S8       | <i>SP</i> + <i>TP</i> + <i>IS</i> & <i>RP</i> | 33.55                                        | 34.11 | 55.66 | 38.18 | 34.13 | 36.70 | -                              | -            | -            | -            | -            | -            |
| S9       | S6 + <i>RP</i>                                | 31.85                                        | 33.00 | 49.82 | 36.12 | 31.77 | 34.52 | 12.23                          | 14.05        | 20.94        | 12.90        | 14.75        | 13.62        |
| S10      | S9 + Parallel WaveGan                         | 30.72                                        | 32.54 | 49.94 | 35.42 | 30.61 | 33.64 | 11.18                          | 13.27        | 19.04        | 12.55        | 12.44        | 12.51        |
| S11      | S9 + BigVGAN                                  | 31.96                                        | 33.33 | 50.05 | 36.14 | 32.13 | 34.67 | 11.29                          | 13.28        | 19.11        | 12.70        | 12.43        | 12.60        |
| S12      | S9 + Vocos                                    | 30.84                                        | 32.14 | 50.03 | 35.04 | 31.33 | 33.68 | 11.27                          | 12.98        | 18.97        | 12.55        | 12.50        | 12.53        |
| S13      | Fusion S9-S11                                 | -                                            | -     | -     | -     | -     | -     | 11.28                          | 12.48        | 18.93        | 12.20        | 12.85        | 12.30        |
| S14      | Fusion all                                    | -                                            | -     | -     | -     | -     | -     | <b>10.87</b>                   | <b>12.27</b> | <b>18.03</b> | <b>11.88</b> | <b>12.21</b> | <b>12.01</b> |

*SP* represents speed perturbation; *TP* represents tempo perturbation; *RP* represents RepeatPart; *IS* represents InsertSilence; & indicates that data augmentation methods are used on top of each other; **Mild**, **Mod.**, and **Sev.** represent the CER for different severities of speech stuttering, corresponding to Mild, Moderate, and Severe; **Conv.** and **Cmd.** represent the CER for conversation scenarios and command word types, respectively; **All** represents the CER for all types.

S10 achieved the best performance, with CER reductions of 0.88% and 1.11% compared to S9. However, BigVGAN and Vocos did not perform as well, possibly due to the limited amount of stuttered speech data available for training the GANs, leading to unstable performance, and the GANs learning fewer stutter features from non-parallel fluent speech data compared to parallel fluent-stuttering speech data.

Finally, we applied Hystoc [37] to combine the results of multiple systems, each incorporating the stutterer's speaker adaptation, for improved performance. System S9, our best submission in the official evaluation, which fused the results of S9-S11, achieved a CER of 12.30%. Post-challenge, we fused all systems, achieving an optimal CER of 12.01%, which represents a significant 7.17% absolute improvement over the official baseline.

## 5.2. Results on StutteringSpeech Challenge Track 2

The StutteringSpeech Challenge employs the CER as a metric for evaluation. The CER results for the official baseline and Top 5 competition systems are presented in Table 3, where our system demonstrates a CER of 12.30% on the Test dataset. It is evident from table that our model outperforms the official baseline model, showcasing a relative enhancement of 35.87% in CER. This improvement can be attributed to the utilization of proficient data augmentation techniques and sophisticated model architecture.

**Table 3:** Results on StutteringSpeech Challenge Track 2

| Team                         | CER (%)      |
|------------------------------|--------------|
| <b>Our System (Rank 1st)</b> | <b>12.30</b> |
| Rank 2nd Team                | 16.85        |
| Rank 3rd Team                | 17.33        |
| Rank 4th Team                | 17.92        |
| Rank 5th Team                | 18.16        |
| ...                          | ...          |
| Official Baseline            | 19.18        |

## 6. CONCLUSION

This paper proposes a series of straightforward yet effective data augmentation methods for stuttering speech recognition, aimed at improving the overall performance of such systems. These personalized data augmentation techniques can flexibly extend the limited amount of stuttering speech training data. Experimental results on the state-of-the-art end-to-end E-Branchformer ASR system demonstrate that combining adversarial and signal-based data augmentation methods significantly improves the generalization performance of ASR models. Future research will focus on refining signal-based front-end data augmentation methods and improving adversarial data augmentation techniques to simulate better data containing richer stuttering speech characteristics.

## 7. REFERENCES

- [1] Shaomei Wu, “The world is designed for fluent people: Benefits and challenges of videoconferencing technologies for people who stutter,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–17.
- [2] Michael P Boyle, “Enacted stigma and felt stigma experienced by adults who stutter,” *Journal of communication disorders*, vol. 73, pp. 50–61, 2018.
- [3] Geoffrey A Coalson, Alexis Crawford, Shanley B Treleaven, Courtney T Byrd, Lauren Davis, Lillian Dang, Jillian Edgerly, and Alison Turk, “Microaggression and the adult stuttering experience,” *Journal of Communication Disorders*, vol. 95, pp. 106180, 2022.
- [4] Colin Lea, Zifang Huang, Jaya Narain, Lauren Tooley, Dianna Yee, Dung Tien Tran, Panayiotis Georgiou, Jeffrey P Bigham, and Leah Findlater, “From user perceptions to technical improvement: Enabling people who stutter to better use speech recognition,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–16.
- [5] Olabanji Shonibare, Xiaosu Tong, and Venkatesh Ravichandran, “Enhancing asr for stuttered speech with limited data using detect and pass,” *arXiv preprint arXiv:2202.05396*, 2022.
- [6] Yi Liu, Yuekang Li, Gelei Deng, Felix Juefei-Xu, Yao Du, Cen Zhang, Chengwei Liu, Yeting Li, Lei Ma, and Yang Liu, “Aster: Automatic speech recognition system accessibility testing for stutterers,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 510–521.
- [7] Xin Zhang, Iván Vallés-Pérez, Andreas Stolcke, Chengzhu Yu, Jasha Droppo, Olabanji Shonibare, Roberto Barra-Chicote, and Venkatesh Ravichandran, “Stutter-tts: Controlled synthesis and improved recognition of stuttered speech,” *arXiv preprint arXiv:2211.09731*, 2022.
- [8] Rong Gong, Hongfei Xue, Lezhi Wang, Xin Xu, Qisheng Li, Lei Xie, Hui Bu, Shaomei Wu, Jiaming Zhou, Yong Qin, et al., “As-70: A mandarin stuttered speech dataset for automatic speech recognition and stuttering event detection,” *arXiv preprint arXiv:2406.07256*, 2024.
- [9] Colin Lea, Vikramjit Mitra, Aparna Joshi, Sachin Kajari, and Jeffrey P Bigham, “Sep-28k: A dataset for stuttering event detection from podcasts with people who stutter,” in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6798–6802.
- [10] Robert L MacDonald, Pan-Pan Jiang, Julie Cattiau, Rus Heywood, Richard Cave, Katie Seaver, Marilyn A Ladewig, Jimmy Tobin, Michael P Brenner, Philip C Nelson, et al., “Disordered speech data collection: Lessons learned at 1 million utterances from project euphonia,” in *Interspeech*, 2021, vol. 2021, pp. 4833–4837.
- [11] Shangkun Huang, Yuxuan Du, Yankai Wang, Jing Deng, and Rong Zheng, “The fosafer system for the icassp2024 in-car multi-channel automatic speech recognition challenge,” in *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*. IEEE, 2024, pp. 5–6.
- [12] Mengzhe Geng, Xurong Xie, Shansong Liu, Jianwei Yu, Shoukang Hu, Xunying Liu, and Helen Meng, “Investigation of data augmentation techniques for disordered speech recognition,” *arXiv preprint arXiv:2201.05562*, 2022.
- [13] Daniel S Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D Cubuk, and Quoc V Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” *arXiv preprint arXiv:1904.08779*, 2019.
- [14] Tom Ko, Vijayaditya Peddinti, Daniel Povey, Michael L Seltzer, and Sanjeev Khudanpur, “A study on data augmentation of reverberant speech for robust speech recognition,” in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 5220–5224.
- [15] Zengrui Jin, Mengzhe Geng, Xurong Xie, Jianwei Yu, Shansong Liu, Xunying Liu, and Helen Meng, “Adversarial data augmentation for disordered speech recognition,” *arXiv preprint arXiv:2108.00899*, 2021.
- [16] John Harvill, Dias Issa, Mark Hasegawa-Johnson, and Changdong Yoo, “Synthesis of new words for improved dysarthric speech recognition on an expanded vocabulary,” in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6428–6432.
- [17] Jennifer Drexler and James Glass, “Combining end-to-end and adversarial training for low-resource speech recognition,” in *2018 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2018, pp. 361–368.
- [18] Helin Wang, Thomas Thebaud, Jesus Villalba, Myra Sydnor, Becky Lammers, Najim Dehak, and Laureano Moro-Velazquez, “Duta-vc: A duration-aware typical-to-atypical voice conversion approach with diffusion

- probabilistic model,” *arXiv preprint arXiv:2306.10588*, 2023.
- [19] Wen-Chin Huang, Bence Mark Halpern, Lester Phillip Violeta, Odette Scharenborg, and Tomoki Toda, “Towards identity preserving normal to dysarthric voice conversion,” in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 6672–6676.
- [20] Wen-Chin Huang, Kazuhiro Kobayashi, Yu-Huai Peng, Ching-Feng Liu, Yu Tsao, Hsin-Min Wang, and Tomoki Toda, “A preliminary study of a two-stage paradigm for preserving speaker identity in dysarthric voice conversion,” *arXiv preprint arXiv:2106.01415*, 2021.
- [21] Vicky Zayats, Mari Ostendorf, and Hannaneh Hajishirzi, “Disfluency detection using a bidirectional lstm,” *arXiv preprint arXiv:1604.03209*, 2016.
- [22] Olabanji Shonibare, Xiaosu Tong, and Venkatesh Ravichandran, “Enhancing asr for stuttered speech with limited data using detect and pass,” *arXiv preprint arXiv:2202.05396*, 2022.
- [23] Colin Lea, Zifang Huang, Jaya Narain, Lauren Tooley, Dianna Yee, Dung Tien Tran, Panayiotis Georgiou, Jeffrey P Bigham, and Leah Findlater, “From user perceptions to technical improvement: Enabling people who stutter to better use speech recognition,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–16.
- [24] Vikramjit Mitra, Zifang Huang, Colin Lea, Lauren Tooley, Sarah Wu, Darren Botten, Ashwini Palekar, Shrinath Thelapurath, Panayiotis Georgiou, Sachin Kawarekar, et al., “Analysis and tuning of a voice assistant system for dysfluent speech,” *arXiv preprint arXiv:2106.11759*, 2021.
- [25] Zengrui et al. Jin, “Personalized adversarial data augmentation for dysarthric and elderly speech recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 413–429, 2024.
- [26] Huimeng et al. Wang, “Enhancing pre-trained asr system fine-tuning for dysarthric speech recognition using adversarial data augmentation,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 12311–12315.
- [27] Ryuichi Yamamoto, Eunwoo Song, and Jae-Min Kim, “Parallel wavegan: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram,” in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6199–6203.
- [28] Sang-gil Lee, Wei Ping, Boris Ginsburg, Bryan Catanzaro, and Sungroh Yoon, “Bigvgan: A universal neural vocoder with large-scale training,” *arXiv preprint arXiv:2206.04658*, 2022.
- [29] Hubert Siuzdak, “Vocos: Closing the gap between time-domain and fourier-based neural vocoders for high-quality audio synthesis,” *arXiv preprint arXiv:2306.00814*, 2023.
- [30] Tom Ko, Vijayaditya Peddinti, Daniel Povey, and Sanjeev Khudanpur, “Audio augmentation for speech recognition,” in *Interspeech*, 2015, vol. 2015, p. 3586.
- [31] Naoyuki Kanda, Ryu Takeda, and Yasunari Obuchi, “Elastic spectral distortion for low resource speech recognition with deep neural networks,” in *2013 IEEE workshop on automatic speech recognition and understanding*. IEEE, 2013, pp. 309–314.
- [32] Werner Verhelst and Marc Roelands, “An overlap-add technique based on waveform similarity (wsola) for high quality time-scale modification of speech,” in *1993 IEEE International Conference on Acoustics, Speech, and Signal Processing*. IEEE, 1993, vol. 2, pp. 554–557.
- [33] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, “Generative adversarial nets,” *Advances in neural information processing systems*, vol. 27, 2014.
- [34] Hui Bu, Jiayu Du, Xingyu Na, Bengu Wu, and Hao Zheng, “Aishell-1: An open-source mandarin speech corpus and a speech recognition baseline,” in *2017 20th conference of the oriental chapter of the international coordinating committee on speech databases and speech I/O systems and assessment (O-COCOSDA)*. IEEE, 2017, pp. 1–5.
- [35] Sachin Mehta, Mohammad Rastegari, Anat Caspi, Linda Shapiro, and Hannaneh Hajishirzi, “Espnet: Efficient spatial pyramid of dilated convolutions for semantic segmentation,” in *Proceedings of the european conference on computer vision (ECCV)*, 2018, pp. 552–568.
- [36] Kwangyoun Kim, Felix Wu, Yifan Peng, Jing Pan, Prashant Sridhar, Kyu J Han, and Shinji Watanabe, “Ebranchformer: Branchformer with enhanced merging for speech recognition,” in *2022 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2023, pp. 84–91.
- [37] Karel Beneš, Martin Kocour, and Lukáš Burget, “Hystoc: Obtaining word confidences for fusion of end-to-end asr systems,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 11276–11280.